WEB SERVICES FOR JAVA EE

Hands-On Lab Workbook

Platform: Java 11 LTS | Tomcat 9.0 | JAX-WS 2.3 | JAX-RS 2.1 | Maven 3.8+
Starter project: ws-lab-starter.zip (import as Maven project)
Labs: 9 Labs — approx. 5 hours total
Level: Intermediate Java Developer

© Training Materials — All Rights Reserved

Lab Overview & Course Outline

Labs in This Workbook

Lab	Title	Section	Topics	Time
1	SOAP Service Scenarios	Sec 1	SOAP vs REST trade-offs, WSDL reading	25 min
2	First JAX-WS Service	Sec 2 & 3	@WebService, wsdlgen, WSDL structure	40 min
3	JAXB and Complex Types	Sec 3	@XmlRootElement, marshal/unmarshal with JAX-WS	35 min
4	Web Service Metadata	Sec 4	@WebMethod, @WebParam, @SOAPBinding, @HandlerChain	35 min
5	WSDL-First Development	Sec 5	wsimport, generated stubs, contract-first	40 min
6	Java-First Development	Sec 5	@WebService, wsgen, publishing with Endpoint	30 min
7	Provider and Dispatch APIs	Sec 6	Provider<SOAPMessage>, Dispatch<Source>, raw XML	40 min
8	SAAJ and JAXP	Sec 7	MessageFactory, SOAPMessage, DOM/StAX parsing	35 min
9	JAX-RS and JAXR Survey	Sec 8 & 9	REST service, UDDI lookup (survey only)	25 min

Starter project setup

1. Unzip ws-lab-starter.zip and import as a Maven project.
2. Run: mvn clean package -DskipTests — build must succeed.
3. Run: mvn test -Dtest=SmokeTest — all baseline tests must PASS.
4. For Tomcat tests: cp target/ws-lab.war $CATALINA_HOME/webapps/
5. For JAX-WS standalone: mvn exec:java -Dexec.mainClass=com.example.ws.HelloEndpoint
6. Verify: curl http://localhost:8080/ws-lab/api/health

Important — JAX-WS in Java 11

JAX-WS was removed from the JDK in Java 9 (JEP 320).
All projects must include the jakarta.xml.ws-api and jaxws-rt Maven dependencies.
These are already declared in the starter pom.xml — do not remove them.
Use the jakarta.jws.* and jakarta.xml.ws.* package namespaces throughout.

© Training Materials — All Rights Reserved

LAB 1

SOAP Web Service Scenarios — Analysis and Design

Section: Section 1

25 min

Objectives

- Identify scenarios where SOAP/JAX-WS is the appropriate technology choice
- Compare SOAP and REST on interoperability, tooling, and contract-first design
- Read a basic WSDL document and identify its principal elements
- Determine the hosting model (Servlet container vs Endpoint.publish) appropriate for a given scenario

Task 1.1 — Technology Selection

For each scenario below, decide whether SOAP/JAX-WS or REST/JAX-RS is the better fit. Justify each decision with at least one concrete reason:

Scenarios

a. A bank exposes a funds-transfer operation to external corporate clients using IBM MQ. Partners require a formal WSDL contract for their code generators.

b. A mobile app needs a lightweight product catalogue API — JSON over HTTPS.

c. A healthcare system integrates with a government portal that mandates WS-Security message signing and SOAP 1.2.

d. An internal microservice exposes metrics data polled every 5 seconds by 50 consumers.

e. A legacy ERP system already exposes WSDL endpoints; your new service must call them.

Your technology choices and justifications:

© Training Materials — All Rights Reserved

Task 1.2 — Read a WSDL

The starter project includes src/main/resources/wsdl/hello.wsdl. Open it and answer:

```
<!-- Examine hello.wsdl and answer the questions below -->
<!-- Key WSDL elements to locate:
     <wsdl:types>       — XSD type definitions
     <wsdl:message>     — message part definitions
     <wsdl:portType>    — abstract operations
     <wsdl:binding>     — protocol/encoding binding
     <wsdl:service>     — concrete endpoint address -->
```

Q1 — WSDL analysis

a. What operation does the service expose?
b. What is the SOAPAction for the sayHello operation?
c. What namespace does the targetNamespace use?
d. What style does the binding declare (document or rpc)?
e. What is the concrete endpoint URL in <wsdl:service>?

Your WSDL analysis:

Task 1.3 — Hosting Model Decision

Q2 — For each deployment requirement, state the hosting model and explain why

a. The service must be deployed inside Tomcat 9 alongside JAX-RS resources.
b. The service is a standalone Java application running on a developer laptop.
c. The service must survive Tomcat restarts without re-running any main() method.
d. The service is a one-off integration test that runs as part of mvn test.

Your hosting model decisions:

Lab 1 Checkpoint

✓ Technology selection completed with justifications
✓ WSDL elements identified (operation, SOAPAction, namespace, style, URL)
✓ Hosting model selected and justified for each scenario

© Training Materials — All Rights Reserved

© Training Materials — All Rights Reserved

LAB 2

First JAX-WS Service — @WebService and WSDL Generation

Section: Sections 2 & 3

40 min

Objectives

- Implement a JAX-WS endpoint using @WebService and @WebMethod
- Publish the endpoint using Endpoint.publish() for standalone testing
- Retrieve and inspect the auto-generated WSDL
- Call the service using wsimport-generated client stubs

Task 2.1 — Implement HelloService

Open src/main/java/com/example/ws/hello/HelloServiceImpl.java. Implement the annotated endpoint:

```
import jakarta.jws.WebService;
import jakarta.jws.WebMethod;
import jakarta.jws.WebParam;
import jakarta.jws.soap.SOAPBinding;

@WebService(
    serviceName      = "HelloService",
    portName         = "HelloPort",
    targetNamespace  = "http://example.com/hello"
)
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT,
             use   = SOAPBinding.Use.LITERAL)
public class HelloServiceImpl implements HelloService {

    @WebMethod(operationName = "sayHello")
    public String sayHello(
            @WebParam(name = "name") String name) {
        // TODO: return "Hello, " + name + "! From JAX-WS."
        throw new UnsupportedOperationException("TODO — Lab 2");
    }

    @WebMethod(operationName = "getTime")
    public String getTime() {
        // TODO: return current time as ISO string
        throw new UnsupportedOperationException("TODO — Lab 2");
    }
}
```

Task 2.2 — Publish the Endpoint

Open HelloEndpoint.java (the starter main class). Publish the service and retrieve the WSDL:

```
public class HelloEndpoint {
    public static void main(String[] args) throws Exception {
        String address = "http://localhost:9090/hello";
        // TODO: Endpoint.publish(address, new HelloServiceImpl())
        System.out.println("Published at: " + address);
        System.out.println("WSDL at:      " + address + "?wsdl");
        System.out.println("Press ENTER to stop...");
        System.in.read();
        // TODO: endpoint.stop()
```

© Training Materials — All Rights Reserved

```
    }
}
```

Run and retrieve the WSDL:

```
mvn exec:java -Dexec.mainClass=com.example.ws.hello.HelloEndpoint

# In another terminal:
curl http://localhost:9090/hello?wsdl
# Paste key sections of the WSDL below
```

Paste the <wsdl:portType> and <wsdl:binding> sections from the generated WSDL:

Task 2.3 — Implement the SEI (Service Endpoint Interface)

Open HelloService.java. Define the SEI with @WebService and the two method signatures that HelloServiceImpl will implement:

```
@WebService(targetNamespace = "http://example.com/hello")
public interface HelloService {
    String sayHello(String name);
    String getTime();
}
```

Q1

What is the difference between the SEI and the implementation class? Which carries the @WebService annotation — the interface, the class, or both? What does each annotation do?

Your answer:

Task 2.4 — Write a Java Client

Write a Java client that calls sayHello without using wsimport — use the Service API directly with the endpoint address:

```
// Direct client using Service.create() + QName
QName serviceName = new QName(
    "http://example.com/hello", "HelloService");
```

© Training Materials — All Rights Reserved

```
QName portName = new QName(
    "http://example.com/hello", "HelloPort");

Service service = Service.create(
    new URL("http://localhost:9090/hello?wsdl"),
    serviceName);
HelloService port = service.getPort(portName, HelloService.class);
System.out.println(port.sayHello("World"));
```

Write your completed client code and paste the output:

Lab 2 Checkpoint

- ✓ HelloServiceImpl.sayHello() returns the correct greeting
- ✓ Endpoint.publish() starts the service on port 9090
- ✓ curl http://localhost:9090/hello?wsdl returns a valid WSDL document
- ✓ Java client receives 'Hello, World! From JAX-WS.'

© Training Materials — All Rights Reserved

LAB 3

JAXB and Complex Types in JAX-WS

Section: Section 3

35 min

Objectives

- Add JAXB-annotated complex types as JAX-WS operation parameters and return values
- Observe how JAX-WS reflects JAXB annotations into the generated WSDL schema
- Test roundtrip serialisation of complex types through a live SOAP call

Task 3.1 — Define JAXB Types

Open OrderRequest.java and OrderResponse.java in the starter project. Add JAXB annotations to map them into the WSDL schema:

```
// OrderRequest.java
@XmlRootElement(name = "orderRequest",
                namespace = "http://example.com/order")
@XmlAccessorType(XmlAccessType.FIELD)
public class OrderRequest {
    @XmlElement(required = true)
    private String customerId;
    @XmlElement(required = true)
    private String productId;
    @XmlElement(required = true)
    private int    quantity;
    // getters and setters
}

// OrderResponse.java
@XmlRootElement(name = "orderResponse",
                namespace = "http://example.com/order")
@XmlAccessorType(XmlAccessType.FIELD)
public class OrderResponse {
    @XmlElement(required = true)
    private String orderId;
    @XmlElement(required = true)
    private String status;
    @XmlElement
    private String message;
    // getters and setters
}
```

Task 3.2 — Add Operation to OrderService

Implement OrderServiceImpl.placeOrder() using the two JAXB types:

```
@WebService(serviceName = "OrderService",
            portName = "OrderPort",
            targetNamespace = "http://example.com/order")
public class OrderServiceImpl implements OrderService {

    @WebMethod(operationName = "placeOrder")
    public OrderResponse placeOrder(
             @WebParam(name = "request") OrderRequest request) {
        // TODO:
        // 1. Validate request.getQuantity() > 0 → set status="REJECTED" if
not
```

© Training Materials — All Rights Reserved

```
        // 2. Generate orderId = "ORD-" + System.currentTimeMillis()
        // 3. Return OrderResponse with orderId, status="ACCEPTED", message
        throw new UnsupportedOperationException("TODO — Lab 3");
    }
}
```

Publish and retrieve the WSDL. Locate the <xs:complexType> entries for OrderRequest and OrderResponse:

```
curl http://localhost:9091/order?wsdl
# Look for <xs:complexType name="orderRequest">
# and <xs:complexType name="orderResponse">
```

Paste the XSD complex type definitions from the generated WSDL:

Task 3.3 — Test with a Raw SOAP Request

Use curl to send a raw SOAP 1.1 message and observe the response:

```
curl -X POST http://localhost:9091/order \
     -H 'Content-Type: text/xml;charset=UTF-8' \
     -H 'SOAPAction: """' \
     -d '<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
    xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:ord="http://example.com/order">
  <soapenv:Body>
    <ord:placeOrder>
      <request>
        <customerId>C001</customerId>
        <productId>PROD-42</productId>
        <quantity>3</quantity>
      </request>
    </ord:placeOrder>
  </soapenv:Body>
</soapenv:Envelope>'
```

Paste the SOAP response envelope here:

© Training Materials — All Rights Reserved

Q1

The SOAP fault structure differs from a normal HTTP 400 response. Send a request with quantity=-1. What does the SOAP fault envelope look like? How does the client know it is a fault?

SOAP fault structure and your observations:

Lab 3 Checkpoint

- ✓ OrderRequest and OrderResponse have correct JAXB annotations
- ✓ Generated WSDL contains xs:complexType for both types
- ✓ curl SOAP request returns an OrderResponse with orderId and status
- ✓ Negative quantity returns a SOAP fault response

© Training Materials — All Rights Reserved

LAB 4

Web Service Metadata Annotations

Section: Section 4

35 min

Objectives

- Apply @WebMethod, @WebParam, @WebResult, and @WebFault to control WSDL generation
- Use @SOAPBinding to switch between Document/Literal and RPC/Literal styles
- Define and throw typed SOAP faults using @WebFault
- Implement a @HandlerChain with a simple logging handler

Task 4.1 — @WebMethod, @WebParam, @WebResult

Annotate the ProductService operations with explicit metadata to control WSDL element names:

```
@WebService(targetNamespace = "http://example.com/product")
public interface ProductService {

    @WebMethod(operationName = "findProductById",
               action = "http://example.com/product/findById")
    @WebResult(name = "product",
               targetNamespace = "http://example.com/product")
    ProductInfo findById(
        @WebParam(name = "productId",
                  targetNamespace = "http://example.com/product")
        String id)
        throws ProductNotFoundException;

    @WebMethod(operationName = "listByCategory")
    @WebResult(name = "products")
    List<ProductInfo> listByCategory(
        @WebParam(name = "category") String category,
        @WebParam(name = "maxResults") int maxResults);

    @WebMethod(exclude = true)     // excluded from WSDL
    void internalAudit(String event);
}
```

Q1

What does @WebMethod(exclude=true) do to the generated WSDL? What happens if you remove the annotation entirely from internalAudit()?

Your answer:

Task 4.2 — Typed SOAP Faults with @WebFault

Define a ProductNotFoundException that produces a typed SOAP fault:

```
// ProductNotFoundFault.java — the fault detail bean
```

© Training Materials — All Rights Reserved

```
@XmlRootElement(name = "productNotFoundFault")
@XmlAccessorType(XmlAccessType.FIELD)
public class ProductNotFoundFault {
    @XmlElement(required = true) private String productId;
    @XmlElement(required = true) private String message;
    // getters and setters
}

// ProductNotFoundException.java — the exception class
@WebFault(name = "ProductNotFoundException",
          faultBean = "com.example.ws.product.ProductNotFoundFault",
          targetNamespace = "http://example.com/product")
public class ProductNotFoundException extends Exception {
    private final ProductNotFoundFault faultInfo;
    public ProductNotFoundException(String message,
                                    ProductNotFoundFault faultInfo) {
        super(message); this.faultInfo = faultInfo;
    }
    public ProductNotFoundFault getFaultInfo() { return faultInfo; }
}

// Throwing the fault in the implementation:
if (product == null) {
    ProductNotFoundFault fault = new ProductNotFoundFault();
    fault.setProductId(id);
    fault.setMessage("Product not found: " + id);
    throw new ProductNotFoundException("Not found", fault);
}
```

Task 4.3 — @SOAPBinding Style Comparison

Publish ProductServiceImpl twice: once with DOCUMENT/LITERAL and once with RPC/LITERAL. Compare the WSDL <binding> sections:

```
// Default (Document/Literal — recommended):
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT,
             use   = SOAPBinding.Use.LITERAL)
public class ProductServiceImpl implements ProductService { ... }

// Alternative (RPC/Literal — for legacy compatibility):
@SOAPBinding(style = SOAPBinding.Style.RPC,
             use   = SOAPBinding.Use.LITERAL)
public class ProductServiceRpcImpl implements ProductService { ... }
```

Compare the <binding> section of the WSDL for DOCUMENT vs RPC style:

Task 4.4 — @HandlerChain Logging Handler

Create a LoggingHandler that prints the SOAPMessage for every inbound and outbound message. Register it with @HandlerChain:

```
// LoggingHandler.java
```

© Training Materials — All Rights Reserved

```
public class LoggingHandler
        implements SOAPHandler<SOAPMessageContext> {

    @Override
    public boolean handleMessage(SOAPMessageContext ctx) {
        boolean outbound = (Boolean) ctx.get(
            MessageContext.MESSAGE_OUTBOUND_PROPERTY);
        System.out.printf("[SOAP %s]%n",
            outbound ? "OUTBOUND" : "INBOUND");
        try {
            ctx.getMessage().writeTo(System.out);
        } catch (Exception e) { e.printStackTrace(); }
        return true;  // continue processing
    }

    @Override
    public boolean handleFault(SOAPMessageContext ctx) {
        System.out.println("[SOAP FAULT]");
        try { ctx.getMessage().writeTo(System.out); }
        catch (Exception e) { e.printStackTrace(); }
        return true;
    }

    @Override public void close(MessageContext ctx) {}
    @Override public Set<QName> getHeaders() { return null; }
}

// handlers.xml (classpath resource):
// <handler-chains>
//   <handler-chain>
//     <handler>
//       <handler-class>com.example.ws.handler.LoggingHandler</handler-class>
//     </handler>
//   </handler-chain>
// </handler-chains>

// On the service class:
// @HandlerChain(file = "handlers.xml")
// public class ProductServiceImpl ...
```

Write your completed LoggingHandler and handlers.xml, and paste the console output from a test call:

Lab 4 Checkpoint

- ✓ WSDL contains correct element names from @WebParam/@WebResult
- ✓ @WebMethod(exclude=true) removes internalAudit from the WSDL
- ✓ Invalid product ID returns a typed SOAP fault with ProductNotFoundFault detail
- ✓ LoggingHandler prints SOAP messages to console for each call

© Training Materials — All Rights Reserved

LAB 5

WSDL-First (Contract-First) Development

Section: Section 5

40 min

Objectives

- Generate Java client and server stubs from an existing WSDL using wsimport
- Implement a service that conforms to a pre-existing WSDL contract
- Understand the relationship between WSDL elements and generated Java classes

Task 5.1 — Examine the Existing WSDL

The starter project includes src/main/resources/wsdl/weather.wsdl — a complete WSDL for a weather data service. Before generating any code, examine it:

Locate these elements in weather.wsdl

1. The targetNamespace
2. The operation name(s) in <portType>
3. The input/output message parts
4. The XSD types used in the messages
5. The concrete endpoint URL in <service>

Your WSDL analysis (targetNamespace, operations, message types, endpoint URL):

Task 5.2 — Generate Stubs with wsimport (jaxws-maven-plugin)

The pom.xml contains a commented-out jaxws-maven-plugin block. Uncomment it and run wsimport:

```
<!-- In pom.xml, uncomment this block: -->
<plugin>
  <groupId>com.sun.xml.ws</groupId>
  <artifactId>jaxws-maven-plugin</artifactId>
  <version>2.3.6</version>
  <executions>
    <execution>
      <goals><goal>wsimport</goal></goals>
      <configuration>
        <wsdlDirectory>src/main/resources/wsdl</wsdlDirectory>
        <wsdlFiles><wsdlFile>weather.wsdl</wsdlFile></wsdlFiles>
        <packageName>com.example.ws.weather.generated</packageName>
        <sourceDestDir>
          ${project.build.directory}/generated-sources/wsimport
        </sourceDestDir>
        <keep>true</keep>
      </configuration>
    </execution>
  </executions>
```

© Training Materials — All Rights Reserved

```
</plugin>

# Then run:
mvn generate-sources
# Generated classes appear in target/generated-sources/wsimport/
find target/generated-sources/wsimport -name '*.java' | sort
```

List all generated Java files and describe the purpose of each:

Task 5.3 — Implement the Service from the Generated SEI

wsimport generates a Service Endpoint Interface. Implement it in WeatherServiceImpl:

```
// WeatherServiceImpl must implement the generated SEI exactly
// (use the generated interface name from Task 5.2):
@WebService(
    endpointInterface = "com.example.ws.weather.generated.WeatherPort",
    serviceName       = "WeatherService",
    portName          = "WeatherPort",
    targetNamespace   = "http://example.com/weather"
)
public class WeatherServiceImpl implements WeatherPort {

    @Override
    public WeatherReport getWeather(String city, String date) {
        // TODO: return a stub WeatherReport
        // Use the generated WeatherReport class (from wsimport)
        WeatherReport report = new WeatherReport();
        report.setCity(city);
        report.setDate(date);
        report.setTemperatureCelsius(21.5);
        report.setCondition("Sunny");
        return report;
    }
}
```

Write your completed WeatherServiceImpl:

© Training Materials — All Rights Reserved

Q1

In contract-first development, who owns the WSDL — the service provider or the consumer? What breaks if the provider changes the WSDL unilaterally?

Your answer:

Lab 5 Checkpoint

- ✓ mvn generate-sources produces Java classes from weather.wsdl
- ✓ WeatherServiceImpl compiles against the generated SEI
- ✓ Endpoint.publish() serves the implementation at the WSDL address
- ✓ A Java client using the generated Service class calls getWeather() successfully

© Training Materials — All Rights Reserved

LAB 6

Java-First Development — wsgen and Endpoint.publish

Section: Section 5

30 min

Objectives

- Publish a JAX-WS service using Endpoint.publish() from a main() method
- Use wsgen to generate portable artifacts (WSDL + XSD) from the implementation class
- Understand when Endpoint.publish() is appropriate vs Servlet container deployment

Task 6.1 — Publish with Endpoint.publish()

Complete the InventoryEndpoint main method to publish InventoryServiceImpl:

```
public class InventoryEndpoint {
    public static void main(String[] args) throws Exception {
        InventoryServiceImpl impl = new InventoryServiceImpl();

        // TODO: Endpoint ep = Endpoint.publish(
        //   "http://localhost:9092/inventory", impl);

        System.out.println("Inventory service started.");
        System.out.println("WSDL: http://localhost:9092/inventory?wsdl");

        // Keep alive until Enter is pressed
        System.in.read();
        // ep.stop();
    }
}

// InventoryServiceImpl to implement:
@WebService(serviceName = "InventoryService",
           portName = "InventoryPort",
           targetNamespace = "http://example.com/inventory")
public class InventoryServiceImpl {

    @WebMethod
    public int getStock(@WebParam(name="sku") String sku) {
        // TODO: return a stub stock level (e.g. sku.hashCode() % 100 + 1)
        throw new UnsupportedOperationException("TODO — Lab 6");
    }

    @WebMethod
    public boolean reserveStock(
            @WebParam(name="sku") String sku,
            @WebParam(name="quantity") int quantity) {
        // TODO: return quantity <= getStock(sku)
        throw new UnsupportedOperationException("TODO — Lab 6");
    }
}
```

```
mvn exec:java -Dexec.mainClass=com.example.ws.inventory.InventoryEndpoint
curl 'http://localhost:9092/inventory?wsdl'
```

© Training Materials — All Rights Reserved

Task 6.2 — Run wsgen to Produce Portable Artifacts

```
# wsgen reads the compiled implementation class and produces:
# - WSDL file suitable for distribution to clients
# - XSD schema files for complex types
# - JAX-WS wrapper classes for wrapped document/literal

mvn compile
wsgen -cp target/classes \
      -wsdl \
      -d target/generated-sources/wsgen \
      -s target/generated-sources/wsgen \
      com.example.ws.inventory.InventoryServiceImpl

ls target/generated-sources/wsgen/
# Expected: InventoryService.wsdl, InventoryService_schema1.xsd
# and wrapper classes (GetStock.java, GetStockResponse.java, etc.)
```

List the wsgen output files and describe their purpose:

Q1

Compare Endpoint.publish() and deploying a WAR to Tomcat. For each of these factors, state which approach is better: clustering, hot-deploy, HTTPS, unit testing, startup speed.

Your comparison:

Lab 6 Checkpoint

- ✓ InventoryServiceImpl.getStock() and reserveStock() are implemented
- ✓ Endpoint.publish() serves the service on port 9092
- ✓ wsgen produces InventoryService.wsdl and schema files
- ✓ The WSDL contains both operations with correct message types

© Training Materials — All Rights Reserved

LAB 7

Provider and Dispatch APIs — Low-Level Message Handling

Section: Section 6

40 min

Objectives

- Implement a Provider<SOAPMessage> to handle raw inbound SOAP messages
- Use Dispatch<Source> to send raw XML to a JAX-WS endpoint
- Understand when low-level APIs are preferred over generated stubs

Task 7.1 — Implement Provider<SOAPMessage>

A Provider<SOAPMessage> is the server-side counterpart to Dispatch — it receives the raw SOAPMessage and returns a raw SOAPMessage without any data binding:

```
import jakarta.xml.ws.*;
import jakarta.xml.soap.*;

@WebServiceProvider(
    serviceName     = "EchoService",
    portName        = "EchoPort",
    targetNamespace = "http://example.com/echo",
    wsdlLocation    = "WEB-INF/wsdl/echo.wsdl"
)
@ServiceMode(value = Service.Mode.MESSAGE)   // MESSAGE = full SOAPMessage
public class EchoProvider implements Provider<SOAPMessage> {

    @Override
    public SOAPMessage invoke(SOAPMessage request) {
        try {
            // TODO:
            // 1. Extract the text content from the request body
            //    SOAPBody body = request.getSOAPBody();
            //    String text = body.getFirstChild().getTextContent();
            // 2. Build a response SOAPMessage
            //    MessageFactory mf = MessageFactory.newInstance();
            //    SOAPMessage response = mf.createMessage();
            //    SOAPBody rb = response.getSOAPBody();
            //    rb.addChildElement("echoResponse").setTextContent("ECHO: "
+ text);
            //    response.saveChanges();
            // 3. return response;
            throw new UnsupportedOperationException("TODO — Lab 7");
        } catch (SOAPException e) {
            throw new WebServiceException(e);
        }
    }
}
```

Task 7.2 — Call with Dispatch<Source>

Write a client that calls the EchoProvider using Dispatch<Source> (raw XML, no generated stubs):

```
QName svcName  = new QName("http://example.com/echo", "EchoService");
QName portName = new QName("http://example.com/echo", "EchoPort");
```

© Training Materials — All Rights Reserved

```
Service service = Service.create(svcName);
service.addPort(portName, SOAPBinding.SOAP11HTTP_BINDING,
    "http://localhost:9090/echo");

Dispatch<Source> dispatch = service.createDispatch(
    portName, Source.class, Service.Mode.PAYLOAD);

// Build raw XML payload
String xml = "<echo xmlns=\"http://example.com/echo\">Hello World</echo>";
Source request = new StreamSource(new StringReader(xml));

// Invoke
Source response = dispatch.invoke(request);

// Parse response
TransformerFactory tf = TransformerFactory.newInstance();
Transformer t = tf.newTransformer();
StringWriter sw = new StringWriter();
t.transform(response, new StreamResult(sw));
System.out.println("Response: " + sw.toString());
```

Write your completed EchoProvider and Dispatch client, and paste the console output:

Q1

Give two scenarios where Provider<SOAPMessage> or Dispatch<Source> is preferable to using generated stubs (wsimport).

Your answer:

Lab 7 Checkpoint

- ✓ EchoProvider returns a SOAPMessage containing "ECHO: Hello World"
- ✓ Dispatch<Source> client successfully calls the provider
- ✓ Mode.MESSAGE vs Mode.PAYLOAD difference understood and noted

© Training Materials — All Rights Reserved

LAB 8

SAAJ and JAXP — Constructing and Parsing SOAP Messages

Section: Section 7

35 min

Objectives

- Build a SOAP message programmatically using SAAJ MessageFactory and SOAPMessage
- Add SOAP headers, body elements, and attachments using the SAAJ API
- Parse a SOAP message body using DOM and StAX (JAXP)
- Send a SAAJ SOAPMessage using SOAPConnection

Task 8.1 — Build a SOAPMessage with SAAJ

Use the MessageFactory and SOAPMessage API to construct a SOAP 1.1 message from scratch:

```
import jakarta.xml.soap.*;

// Create a SOAP 1.1 message
MessageFactory mf =
MessageFactory.newInstance(SOAPConstants.SOAP_1_1_PROTOCOL);
SOAPMessage message = mf.createMessage();
SOAPEnvelope env = message.getSOAPPart().getEnvelope();

// TODO: Add a custom header
SOAPHeader header = env.getHeader();
SOAPHeaderElement correlId = header.addHeaderElement(
    env.createName("CorrelationId","ws","http://example.com/ws"));
correlId.addTextNode("CORR-" + System.currentTimeMillis());

// TODO: Add the body
SOAPBody body = env.getBody();
SOAPBodyElement orderElem = body.addBodyElement(
    env.createName("placeOrder","ord","http://example.com/order"));
orderElem.addChildElement("customerId").addTextNode("C001");
orderElem.addChildElement("productId").addTextNode("PROD-99");
orderElem.addChildElement("quantity").addTextNode("5");

message.saveChanges();

// Print to stdout
System.out.println("=== Built SOAP message ===");
message.writeTo(System.out);
```

Paste the constructed SOAP message here (after writeTo(System.out)):

© Training Materials — All Rights Reserved

Task 8.2 — Parse with DOM (JAXP)

Given the response SOAPMessage from OrderService, parse the orderId and status fields using DOM:

```
// Assume 'response' is a SOAPMessage received from the service
SOAPBody rb = response.getSOAPBody();

// DOM traversal
org.w3c.dom.Node orderResp = rb.getFirstChild();
String orderId = null; String status = null;

org.w3c.dom.NodeList children = orderResp.getChildNodes();
for (int i = 0; i < children.getLength(); i++) {
    org.w3c.dom.Node child = children.item(i);
    if ("orderId".equals(child.getLocalName()))
        orderId = child.getTextContent();
    if ("status".equals(child.getLocalName()))
        status  = child.getTextContent();
}
System.out.println("Parsed orderId=" + orderId + ", status=" + status);
```

Task 8.3 — Parse with StAX (XMLStreamReader)

Repeat the parse in Task 8.2 using StAX — the streaming alternative to DOM:

```
// Convert the SOAPBody to an InputStream
byte[] bodyBytes = bodyToBytes(response);   // helper provided in starter

XMLInputFactory xif = XMLInputFactory.newInstance();
xif.setProperty(XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES, false);
xif.setProperty(XMLInputFactory.SUPPORT_DTD, false);  // XXE prevention
XMLStreamReader xsr = xif.createXMLStreamReader(
    new java.io.ByteArrayInputStream(bodyBytes));

String orderId = null; String status = null;
String current = null;
while (xsr.hasNext()) {
    int event = xsr.next();
    if (event == XMLStreamConstants.START_ELEMENT)
        current = xsr.getLocalName();
    if (event == XMLStreamConstants.CHARACTERS) {
        if ("orderId".equals(current)) orderId = xsr.getText().trim();
        if ("status".equals(current))  status  = xsr.getText().trim();
    }
}
xsr.close();
System.out.println("StAX parsed orderId=" + orderId + ", status=" + status);
```

Q1

When would you choose DOM over StAX for SOAP body parsing? When would you choose StAX? Consider memory usage, random access needs, and document size.

Your answer:

© Training Materials — All Rights Reserved

Lab 8 Checkpoint

- ✓ SAAJ constructs a valid SOAP 1.1 message with header and body
- ✓ DOM parser extracts orderId and status from the response
- ✓ StAX parser produces identical results with streaming approach
- ✓ XXE properties set on XMLInputFactory before parsing

© Training Materials — All Rights Reserved

LAB 9

JAX-RS REST Services and JAXR Survey

Section: Sections 8 & 9

25 min

Objectives

- Review how a JAX-RS REST resource sits alongside JAX-WS SOAP services in one WAR
- Compare SOAP and REST calling patterns from a client perspective
- Survey JAXR (Java API for XML Registries) and understand its role in UDDI

Task 9.1 — JAX-RS Alongside JAX-WS in One WAR

The starter project deploys both a JAX-WS endpoint (via sun-jaxws.xml) and a JAX-RS application. Verify both work from the same WAR and answer the questions:

```
# JAX-RS health endpoint
curl http://localhost:8080/ws-lab/api/health

# JAX-RS product catalogue (REST)
curl http://localhost:8080/ws-lab/api/products
curl http://localhost:8080/ws-lab/api/products/P001

# JAX-WS SOAP product service (SOAP)
curl http://localhost:8080/ws-lab/ProductService?wsdl
```

Q1 — REST vs SOAP caller comparison

For each statement, state whether it applies to REST, SOAP, or both:

a. Clients must parse a WSDL before making their first call.
b. The request body is always XML.
c. Transport-level security (TLS) is used by default for HTTPS.
d. Business errors can be represented as typed fault messages.
e. The client can use a plain curl command with a JSON body.
f. Tooling can generate a strongly-typed client from a service contract.

Your REST vs SOAP comparison answers (a–f):

Task 9.2 — JAXR Survey

JAXR (Java API for XML Registries, JSR-93) was the JEE standard API for accessing UDDI (Universal Description, Discovery, and Integration) and ebXML registries. It is now largely deprecated but may appear in legacy systems.

© Training Materials — All Rights Reserved

JAXR Quick Reference (read-only — no coding required)

Key JAXR interfaces:

ConnectionFactory — creates connections to a registry endpoint
Connection — session with the registry
RegistryService — entry point for BusinessQueryManager and BusinessLifeCycleManager
BusinessQueryManager — find/search operations on the registry
Organization — a registered business entity
Service — a service offered by an Organization
ServiceBinding — the concrete access point (URL) of a Service

Typical UDDI lookup flow:

1. ConnectionFactory.newInstance() with UDDI inquiry URL
2. connection.getRegistryService().getBusinessQueryManager()
3. bqm.findOrganizations(...) → BulkResponse
4. Iterate Organizations → Services → ServiceBindings
5. Use binding.getAccessURI() as the service endpoint address

Q2 — JAXR and modern alternatives

a. UDDI registries were intended to solve what problem for web services?
b. What technology has largely replaced UDDI for REST service discovery?
c. Why is JAXR marked as deprecated in recent JEE specifications?
d. If you inherited code that uses JAXR, what would be your migration strategy?

Your JAXR survey answers:

Lab 9 Checkpoint

✓ Both JAX-RS and JAX-WS endpoints respond from the same WAR
✓ REST vs SOAP comparison completed for all six statements
✓ JAXR/UDDI survey questions answered

© Training Materials — All Rights Reserved

Model Answers — Key Code

Reference implementations. Consult only after attempting each lab.

Lab 2 — HelloServiceImpl

```
@WebMethod(operationName = "sayHello")
public String sayHello(@WebParam(name = "name") String name) {
    return "Hello, " + name + "! From JAX-WS.";
}

@WebMethod(operationName = "getTime")
public String getTime() {
    return java.time.LocalDateTime.now().toString();
}
```

Lab 3 — OrderServiceImpl.placeOrder()

```
public OrderResponse placeOrder(OrderRequest request) {
    OrderResponse response = new OrderResponse();
    if (request.getQuantity() <= 0) {
        response.setStatus("REJECTED");
        response.setMessage("Quantity must be positive");
        return response;
    }
    response.setOrderId("ORD-" + System.currentTimeMillis());
    response.setStatus("ACCEPTED");
    response.setMessage("Order placed for product " +
request.getProductId());
    return response;
}
```

Lab 7 — EchoProvider.invoke()

```
public SOAPMessage invoke(SOAPMessage request) {
    try {
        String text = request.getSOAPBody()
            .getFirstChild().getTextContent();
        MessageFactory mf = MessageFactory.newInstance();
        SOAPMessage response = mf.createMessage();
        response.getSOAPBody()
            .addChildElement("echoResponse")
            .setTextContent("ECHO: " + text);
        response.saveChanges();
        return response;
    } catch (SOAPException e) {
        throw new WebServiceException(e);
    }
}
```

Lab 6 — InventoryServiceImpl

© Training Materials — All Rights Reserved

```
@WebMethod
public int getStock(@WebParam(name="sku") String sku) {
    return Math.abs(sku.hashCode() % 100) + 1;
}

@WebMethod
public boolean reserveStock(
        @WebParam(name="sku") String sku,
        @WebParam(name="quantity") int quantity) {
    return quantity > 0 && quantity <= getStock(sku);
}
```

© Training Materials — All Rights Reserved